Explicação Detalhada das Alíneas do Exercício (Z3 SMT Solver)

Este documento descreve a modelação e a verificação formal do algoritmo de **multiplicação por somas e deslocamentos bit a bit** implementado como um Sistema de Transição de Estados (FOTS) utilizando o Z3 SMT Solver.

Alínea A: Modelação do Sistema (FOTS e Relação de Transição)

- **Objetivo:** Definir a infraestrutura base do autómato e a sua relação de transição.
- **O que o código faz:**
  - Declara as variáveis de estado de n bits (`x`, `y`, `z`, `a`, `b`) e o ponteiro de instrução (`pc`), bem como as respetivas variáveis para o próximo estado (`x1`, `y1`, `z1`, `pc1`, etc.).
  - Define os estados de controlo possíveis: `INIT`, `SKIP`, `LEFT`, `RIGHT`, `STOP` e `ERROR`.
  - Modela as condições de *overflow* sem sinal (`overflow_left` e `overflow_right`) usando funções nativas do Z3 (`BVMulNoOverflow`, `BVAddNoOverflow`).
  - Constrói todas as transições discretas (`t_init`, `t_stop`, `t_left`, `t_right`, `t_left_ok`, `t_right_ok`, `t_left_error`, `t_right_error`) e agrupa-as numa única **relação de transição** (`transicao`).
- **Verificação no Z3:** Testa a satisfatibilidade do estado inicial (`resultado_a1`) e da relação de transição (`resultado_a2`) para garantir que o modelo não é trivialmente insatisfatível.

Alínea B: Validação do Invariante por Indução

- **Objetivo:** Provar que a propriedade fundamental do algoritmo ($x \times y + z = a \times b$) é um invariante mantido ao longo de toda a execução.
- **O que o código faz:**
  - Define a função `invariante_b`, que assegura que o estado de controlo é válido, que as fases de deslocamento (`LEFT`/`RIGHT`) respeitam a paridade de `y`, e que a igualdade algébrica é mantida (exceto no estado inicial `INIT` onde as variáveis ainda não foram inicializadas).
- **Verificação no Z3:** Utiliza a indução matemática simples (`base_b` e `passo_b`). O objetivo é que ambos os testes retornem `z3.unsat`, provando que nenhum estado inicial viola o invariante e que nenhuma transição válida consegue quebrar a propriedade.

Alínea C: Segurança e Controlo de Overflow

- **Objetivo:** Provar que o programa é seguro e que o estado de erro (`ERROR`) é inacessível, restringindo o tamanho das entradas.
- **O que o código faz:**

- Introduz restrições de entradas pequenas (`entradas_pequenas`) limitadas a $2^{n/2-1}$ para evitar cortes de bits devidos à aritmética modular de n bits.
- Utiliza a função de extensão de bits `ZeroExt` (`ext`) para alargar os vetores de bits durante o cálculo da igualdade, garantindo precisão matemática exata sem perda de *carry*.
- Define `invariante_c`, excluindo explicitamente o estado `ERROR`.

- **Verificação no Z3:** Executa a 1-indução (`base_c` e `passo_c`) para confirmar que o sistema permanece estritamente seguro e dentro dos limites definidos.

Alínea D: Terminação e Animação (Sem Deadlocks)

- **Objetivo:** Garantir que o programa termina sempre (não entra em loops infinitos) e que não existem bloqueios inesperados (*deadlocks*).
- **O que o código faz:**
  - **Função de Ranking:** Cria uma métrica bem-fundada (`ranking`) combinando o ponteiro de instrução `pc` com o valor numérico de `y` (`z3.BV2Int`), que diminui estritamente a cada passo do algoritmo.
  - **Verificação de Terminação:** Valida que o ranking nunca é negativo (`resultado_d1`) e que decresce estritamente nas transições válidas (`resultado_d2`).
  - **Animação (Sem Deadlock):** Usa quantificadores existenciais (`z3.Exists`) para verificar se todos os estados não finais (`nao_final`) possuem pelo menos um sucessor válido (`resultado_d3`), e se os estados finais (`STOP`, `ERROR`) agem corretamente como sumidouros estáveis (`resultado_d4`).

Alínea E: Correção Funcional no Estado Final (`STOP`)

- **Objetivo:** Provar que, quando o programa atinge o estado de sucesso (`STOP`), o resultado final guardado em `z` é exatamente o produto correto ($a \times b$).
- **O que o código faz:**
  - Define um invariante auxiliar (`invariante_j`) para provar que o estado `STOP` só é alcançado quando `y == 0`.

- **Verificação no Z3:** Utiliza os invariantes anteriores (`prop_b` e `prop_c`) para verificar que, no estado `STOP`, a condição `z != a * b` é insatisfatível, tanto em aritmética modular de 2^n (`resultado_e1`) como em inteiros alargados para entradas pequenas (`resultado_e2`).

Alínea F: k-Indução com k Variável

- **Objetivo:** Testar a robustez da verificação utilizando k-indução genérica e encontrar o menor k necessário para provar propriedades simples (como `pc != ERROR`).
- **O que o código faz:**
  - Implementa a função de k-indução genérica (`k_inducao`) que desdobra traços de comprimento k utilizando substituições de variáveis indexadas (`z3.substitute`).
  - Adiciona uma relação de sumidouro para estabilizar os estados finais (`STOP` e `ERROR`) durante a análise de traços longos.
  - Executa um ciclo que aumenta k até 60 ou até encontrar uma prova válida.

- Executa um ciclo iterativo de $k = 1$ até 60 para testar a propriedade de segurança simples e determinar o menor limiar k (`k_min`) em que a indução fecha com sucesso (`unsat`).

```
========== ALINEA A ==========
Estado inicial: sat
Relacao de transicao: sat

========== ALINEA B ==========
Caso base: unsat
Passo indutivo: unsat
Invariante provado por 1-inducao.

========== ALINEA C ==========
Limite: 16
Caso base: unsat
Passo indutivo: unsat
Seguranca provada por 1-inducao.
O estado ERROR e inacessivel.

========== ALINEA D ==========
Ranking negativo: unsat
Ranking nao decrescente: unsat
Terminacao provada.

========== ALINEA D (ANIMACAO) ==========
Deadlock fora dos finais: unsat
Finais com sucessor: unsat
Animacao provada: o programa termina para quaisquer a, b.

========== ALINEA D (ANIMACAO) ==========
Deadlock fora dos finais: unsat
Finais com sucessor: unsat
Animacao provada: o programa termina para quaisquer a, b.

========== ALINEA E (CORRECAO) ==========
J - Caso base: unsat
J - Passo indutivo: unsat
STOP e z != a*b (mod 2^n): unsat
STOP e z != a*b (inteiros, a,b pequenos): unsat
Correcao provada: ao chegar a STOP, z = a*b.

========== ALINEA F (K-INDUCAO) ==========
Cruzada (invariante C, k=1): unsat unsat
k= 1  base: unsat  passo: sat
k= 2  base: unsat  passo: sat
k= 3  base: unsat  passo: sat
k= 4  base: unsat  passo: sat
k= 5  base: unsat  passo: sat
k= 6  base: unsat  passo: sat
k= 7  base: unsat  passo: sat
k= 8  base: unsat  passo: sat
k= 9  base: unsat  passo: sat
k=10  base: unsat  passo: sat
k=11  base: unsat  passo: sat
k=12  base: unsat  passo: sat
k=13  base: unsat  passo: sat
k=14  base: unsat  passo: sat
k=15  base: unsat  passo: sat
k=16  base: unsat  passo: sat
k=17  base: unsat  passo: sat
```

```
k=18  base: unsat  passo: sat
k=19  base: unsat  passo: sat
k=20  base: unsat  passo: sat
k=21  base: unsat  passo: sat
k=22  base: unsat  passo: sat
k=23  base: unsat  passo: sat
k=24  base: unsat  passo: sat
k=25  base: unsat  passo: sat
k=26  base: unsat  passo: sat
k=27  base: unsat  passo: sat
k=28  base: unsat  passo: sat
k=29  base: unsat  passo: sat
k=30  base: unsat  passo: sat
k=31  base: unsat  passo: sat
k=32  base: unsat  passo: unsat
pc != ERROR e 32-indutivo (menor k testado).

========== RESULTADO FINAL ==========
Alinea A: SATISFAZIVEL
Alinea B: PROVADA
Alinea C: PROVADA
Alinea D: PROVADA
Animacao: PROVADA
Alinea E: PROVADA
Alinea F: PROVADA

Todas as verificacoes passaram.
```

Documentação e Entrega

Porque escolhemos o Z3 SMT Solver?

Decidimos usar o **Z3 Solver** para modelar e verificar este trabalho prático de *Multiplicação por Somas e Deslocamentos Bit a Bit como FOTS / FSM Simbólica*. A lógica de primeira ordem e teorias de *BitVectors* foram a melhor opção porque nos permitiram definir formalmente regras de transição complexas, estados e condições de overflow de forma declarativa e matematicamente rigorosa.

Porque é que a Indução, o Ranking e a Extensão de Bits são importantes?

Ao utilizarmos a indução simples e k-indução para validar invariantes e ao empregar funções de ranking bem-fundadas (ligando o ponteiro de instrução `pc` ao valor numérico de `y`), conseguimos provar propriedades complexas de segurança e terminação. Além disso, o uso de extensões de bits (`ZeroExt`) permitiu evitar a perda de bits por *overflow* aritmético modular, garantindo a correção exata do produto.

Uso de Ferramentas LLM (Registo de Diálogo)

O link para o diálogo integral com o LLM está disponível aqui: https://share.gemini.google/7m6SgyzkoDHi

Durante o desenvolvimento, o uso do LLM focou-se em três pilares avaliados:

1. **Adequação e Modelação Simbólica:** Estruturação da relação de transição do autómato (`transicao`), estados de controlo (`INIT`, `SKIP`, `LEFT`, `RIGHT`, `STOP`, `ERROR`) e gestão de *overflows* sem sinal (`BVMulNoOverflow`, `BVAddNoOverflow`).

2. **Resolução de Problemas Específicos do Framework:** Resolução de conflitos de âmbito de variáveis e instanciação dinâmica de estados no Marimo e no Z3 (especialmente na construção de k-indução com k variável e substituição de vetores).
3. **Suporte à Depuração:** O modelo foi usado como apoio à análise de erros lógicos (como a validação de invariantes nas alíneas C, D e F), sendo a solução final integrada e validada de forma autónoma.